LAKSHMI NARAIN COLLEGE OF TECHNOLOGY

Department of Computer Science & Engineering

ASSIGNMENT REPORT

Submitted By

VARUN JUNEJA

College	**LNCT**
Enrollment Number	**0103CS231449**
Superset ID	**7809995**
Email	**varunjuneja1905@gmail.com**

Academic Session 2025 – 2026

Exercise 1: Implementing the Singleton Pattern

Pattern Category: Creational Pattern

Aim: To implement the Singleton design pattern in Java by ensuring a Logger utility class has only one instance throughout the application's lifecycle, for consistent logging.

Concept: Ensures a class has only one instance and provides a single, global point of access to it.

Scenario: A logging utility class is needed in the application. To guarantee consistent log output, this Logger class must be restricted to exactly one instance throughout the application's lifecycle, no matter how many parts of the program request it.

Steps Performed

1. Created a new Java project named SingletonPatternExample.
2. Defined a Logger class containing a private static instance of itself.
3. Made the constructor of Logger private so it cannot be instantiated from outside the class.
4. Provided a public static getInstance() method that returns the single shared instance.
5. Implemented the Singleton pattern using eager initialization, which the JVM guarantees is thread-safe.
6. Created a SingletonTest class to verify that two separate requests for Logger return the exact same object, and used it to record several log messages.

Project / Files Created

- SingletonPatternExample/src/Logger.java
- SingletonPatternExample/src/SingletonTest.java

Source Code

Logger.java

```
/**
 * Logger.java
 * Singleton Design Pattern Implementation
 *
 * Ensures that only ONE instance of the Logger class exists throughout
 * the application lifecycle, providing a single, consistent point for
 * writing log messages.
 */
public class Logger {

    // Step 2a: private static instance of itself (eagerly created, thread-safe)
    private static final Logger instance = new Logger();

    // counter to prove the same object is reused everywhere
    private int logCount = 0;
```

```
    // Step 2b: private constructor prevents external instantiation
    private Logger() {
        System.out.println("[Logger] A new Logger instance has been created. Hash = " +
this.hashCode());
    }

    // Step 2c: public static accessor method
    public static Logger getInstance() {
        return instance;
    }

    public void log(String message) {
        logCount++;
        System.out.println("[LOG #" + logCount + "] " + message);
    }

    public int getLogCount() {
        return logCount;
    }
}
```

SingletonTest.java

```
/**
 * SingletonTest.java
 * Test class that verifies that only ONE instance of Logger
 * is created and shared across the entire application.
 */
public class SingletonTest {
    public static void main(String[] args) {
        System.out.println("=== Singleton Pattern Test: Logger ===\n");

        // Request the Logger instance from two different "parts" of the app
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        logger1.log("Application started.");
        logger2.log("User authentication module initialized.");
        logger1.log("Database connection established.");

        System.out.println("\nlogger1 hashcode : " + logger1.hashCode());
        System.out.println("logger2 hashcode : " + logger2.hashCode());

        if (logger1 == logger2) {
            System.out.println("\nSUCCESS: logger1 and logger2 reference the SAME
instance.");
        } else {
            System.out.println("\nFAILURE: Singleton pattern violated - two different
instances exist!");
        }

        System.out.println("Total log entries recorded by the single Logger instance: " +
logger1.getLogCount());
```

```
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and run from the command line as shown below: "javac *.java" followed by "java SingletonTest". The real console output is captured in the screenshot below.

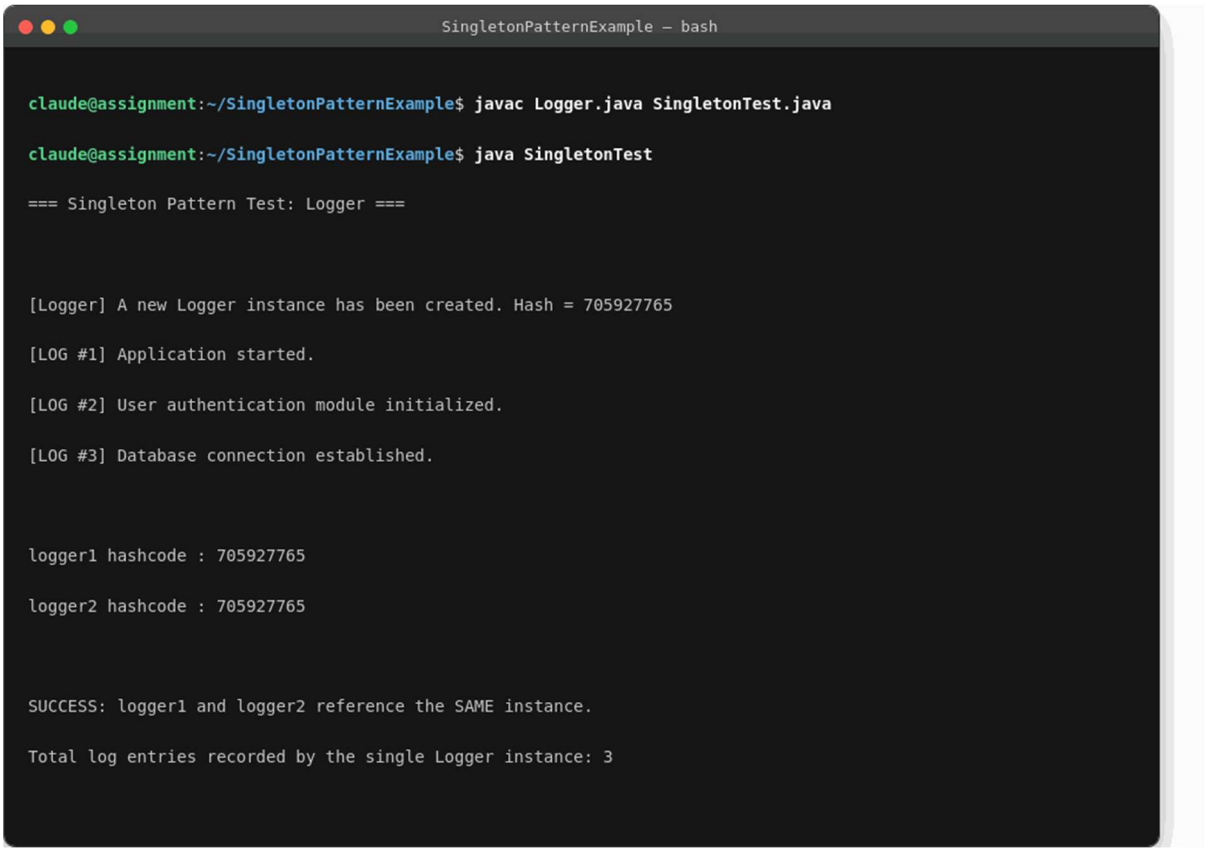

Figure 1: Compilation and execution output - SingletonPatternExample

Output / Result

The console output shows that logger1 and logger2 report the identical hash code (705927765), and the explicit equality check prints “SUCCESS: logger1 and logger2 reference the SAME instance.” This confirms that, regardless of how many times getInstance() is called, only one Logger object is ever created in memory.

Exercise 2: Implementing the Factory Method Pattern

Pattern Category: Creational Pattern

Aim: To implement the Factory Method design pattern in Java for a document management system that must create different types of documents (Word, PDF, Excel).

Concept: Defines an interface for creating an object, but lets subclasses decide which concrete class to instantiate.

Scenario: A document management system needs to create different types of documents - Word, PDF, and Excel - without the client code being tied to the concrete class of each document type.

Steps Performed

1. Created a new Java project named FactoryMethodPatternExample.
2. Defined a Document interface implemented by every document type.
3. Implemented the concrete classes WordDocument, PdfDocument, and ExcelDocument, each providing its own open() and save() behaviour.
4. Implemented the Factory Method: created an abstract DocumentFactory class declaring the abstract method createDocument().
5. Created the concrete factory classes WordDocumentFactory, PdfDocumentFactory, and ExcelDocumentFactory, each extending DocumentFactory and overriding createDocument() to return its corresponding document type.
6. Created a FactoryMethodTest class to demonstrate creating all three document types purely through their respective factories.

Project / Files Created

- FactoryMethodPatternExample/src/Document.java
- FactoryMethodPatternExample/src/WordDocument.java
- FactoryMethodPatternExample/src/PdfDocument.java
- FactoryMethodPatternExample/src/ExcelDocument.java
- FactoryMethodPatternExample/src/DocumentFactory.java
- FactoryMethodPatternExample/src/WordDocumentFactory.java
- FactoryMethodPatternExample/src/PdfDocumentFactory.java
- FactoryMethodPatternExample/src/ExcelDocumentFactory.java
- FactoryMethodPatternExample/src/FactoryMethodTest.java

Source Code

Document.java

```
/** Document.java - common interface implemented by every document type. */
```

```
public interface Document {
    void open();
    void save();
}
```

WordDocument.java

```
public class WordDocument implements Document {
    public void open() { System.out.println("Opening a WORD document (.docx)..."); }
    public void save() { System.out.println("Saving WORD document with Word formatting
engine."); }
}
```

PdfDocument.java

```
public class PdfDocument implements Document {
    public void open() { System.out.println("Opening a PDF document (.pdf)..."); }
    public void save() { System.out.println("Saving PDF document, flattening to portable
format."); }
}
```

ExcelDocument.java

```
public class ExcelDocument implements Document {
    public void open() { System.out.println("Opening an EXCEL document (.xlsx)..."); }
    public void save() { System.out.println("Saving EXCEL document, recalculating
formulas."); }
}
```

DocumentFactory.java

```
/**
 * DocumentFactory.java
 * Abstract Creator class declaring the Factory Method createDocument().
 * Subclasses decide which concrete Document subclass to instantiate.
 */
public abstract class DocumentFactory {

    // The Factory Method
    public abstract Document createDocument();

    // A template operation that uses the factory method
    public Document openNewDocument() {
        Document doc = createDocument();
        doc.open();
        return doc;
    }
}
```

WordDocumentFactory.java

```
public class WordDocumentFactory extends DocumentFactory {
    public Document createDocument() { return new WordDocument(); }
}
```

PdfDocumentFactory.java

```
public class PdfDocumentFactory extends DocumentFactory {
    public Document createDocument() { return new PdfDocument(); }
}
```

ExcelDocumentFactory.java

```
public class ExcelDocumentFactory extends DocumentFactory {
    public Document createDocument() { return new ExcelDocument(); }
}
```

FactoryMethodTest.java

```
/**
 * FactoryMethodTest.java
 * Demonstrates creating different document types via their factories,
 * without the client ever calling "new WordDocument()" etc. directly.
 */
public class FactoryMethodTest {
    public static void main(String[] args) {
        System.out.println("=== Factory Method Pattern Test: Document Management System
===\n");

        DocumentFactory[] factories = {
            new WordDocumentFactory(),
            new PdfDocumentFactory(),
            new ExcelDocumentFactory()
        };

        for (DocumentFactory factory : factories) {
            System.out.println("Using factory: " + factory.getClass().getSimpleName());
            Document doc = factory.openNewDocument();
            doc.save();
            System.out.println("Created instance type: " + doc.getClass().getSimpleName());
            System.out.println("-----------------------------------------------------");
        }

        System.out.println("\nAll document types were created through their respective");
        System.out.println("Factory Method, decoupling client code from concrete classes.");
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and run from the command line as shown below: "javac *.java" followed by "java FactoryMethodTest". The real console output is captured in the screenshot below.

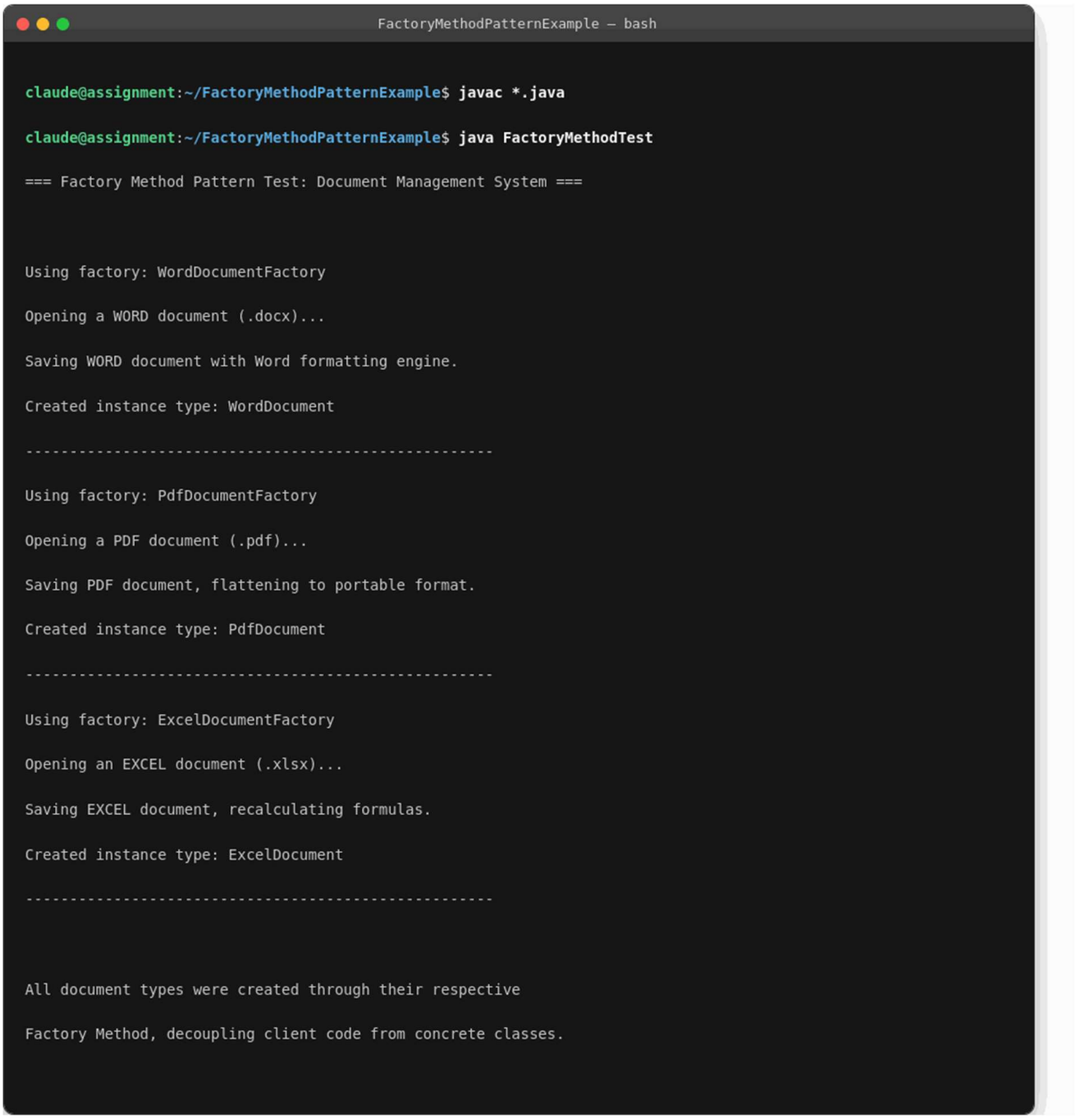

Figure 2: Compilation and execution output - FactoryMethodPatternExample

Output / Result

The output shows that each factory (WordDocumentFactory, PdfDocumentFactory, ExcelDocumentFactory) produced the matching concrete Document subtype and printed its own open/save behaviour. The client (FactoryMethodTest) never wrote “new WordDocument()” directly - confirming that object-creation logic is fully encapsulated inside the factories, as required by the Factory Method pattern.

Exercise 3: Implementing the Builder Pattern

Pattern Category: Creational Pattern

Aim: To implement the Builder design pattern in Java for constructing a complex Computer object that has multiple optional parts.

Concept: Separates the construction of a complex object from its representation, so the same construction process can create different configurations.

Scenario: A system needs to create complex Computer objects that each have several optional parts (CPU, RAM, Storage, GPU, Wi-Fi, Bluetooth). The Builder pattern is used to manage this multi-step construction process cleanly, instead of using a constructor with many parameters.

Steps Performed

1. Created a new Java project named BuilderPatternExample.
2. Defined the Computer product class with attributes CPU, RAM, Storage, GPU, Wi-Fi, and Bluetooth.
3. Implemented a static nested Builder class inside Computer, with fluent methods to set each attribute.
4. Provided a build() method inside the Builder class that returns a fully constructed, immutable Computer instance.
5. Ensured the Computer class has a private constructor that accepts the Builder as its only parameter, so a Computer can only be created through the Builder.
6. Created a BuilderTest class to demonstrate assembling three different Computer configurations (an office PC, a gaming PC, and a budget PC) using the same Builder API.

Project / Files Created

- BuilderPatternExample/src/Computer.java
- BuilderPatternExample/src/BuilderTest.java

Source Code

Computer.java

```
/**
 * Computer.java
 * Product class with many optional attributes. The Builder Pattern lets us
 * construct different configurations of Computer step by step, without
 * a telescoping constructor.
 */
public class Computer {
    // Required
    private final String CPU;
    private final String RAM;
    // Optional
```

```
    private final String storage;
    private final String GPU;
    private final boolean hasWifi;
    private final boolean hasBluetooth;

    // Step 4: private constructor takes the Builder as its only parameter
    private Computer(Builder builder) {
        this.CPU = builder.CPU;
        this.RAM = builder.RAM;
        this.storage = builder.storage;
        this.GPU = builder.GPU;
        this.hasWifi = builder.hasWifi;
        this.hasBluetooth = builder.hasBluetooth;
    }

    @Override
    public String toString() {
        return "Computer Configuration:\n" +
               "  CPU       : " + CPU + "\n" +
               "  RAM       : " + RAM + "\n" +
               "  Storage   : " + (storage == null ? "Not specified" : storage) + "\n" +
               "  GPU       : " + (GPU == null ? "Integrated Graphics" : GPU) + "\n" +
               "  Wi-Fi     : " + (hasWifi ? "Yes" : "No") + "\n" +
               "  Bluetooth : " + (hasBluetooth ? "Yes" : "No");
    }

    // Step 3: static nested Builder class
    public static class Builder {
        private final String CPU;
        private final String RAM;
        private String storage;
        private String GPU;
        private boolean hasWifi;
        private boolean hasBluetooth;

        public Builder(String CPU, String RAM) {
            this.CPU = CPU;
            this.RAM = RAM;
        }

        public Builder storage(String storage) { this.storage = storage; return this; }
        public Builder GPU(String GPU) { this.GPU = GPU; return this; }
        public Builder wifi(boolean hasWifi) { this.hasWifi = hasWifi; return this; }
        public Builder bluetooth(boolean hasBluetooth) { this.hasBluetooth = hasBluetooth;
return this; }

        // build() returns the fully constructed, immutable Computer instance
        public Computer build() {
            return new Computer(this);
        }
    }
}
```

BuilderTest.java

```
/**
 * BuilderTest.java
 * Demonstrates building several different Computer configurations
 * using the fluent Builder API.
 */
public class BuilderTest {
    public static void main(String[] args) {
        System.out.println("=== Builder Pattern Test: Computer Assembly ===\n");

        Computer officePC = new Computer.Builder("Intel i5-13400", "16GB DDR4")
                .storage("512GB SSD")
                .wifi(true)
                .build();

        Computer gamingPC = new Computer.Builder("AMD Ryzen 9 7950X", "32GB DDR5")
                .storage("2TB NVMe SSD")
                .GPU("NVIDIA RTX 4090")
                .wifi(true)
                .bluetooth(true)
                .build();

        Computer budgetPC = new Computer.Builder("Intel Celeron N4020", "4GB DDR4").build();

        System.out.println("---- Office PC ----");
        System.out.println(officePC);
        System.out.println("\n---- Gaming PC ----");
        System.out.println(gamingPC);
        System.out.println("\n---- Budget PC ----");
        System.out.println(budgetPC);
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and run from the command line as shown below: "javac *.java" followed by "java BuilderTest". The real console output is captured in the screenshot below.

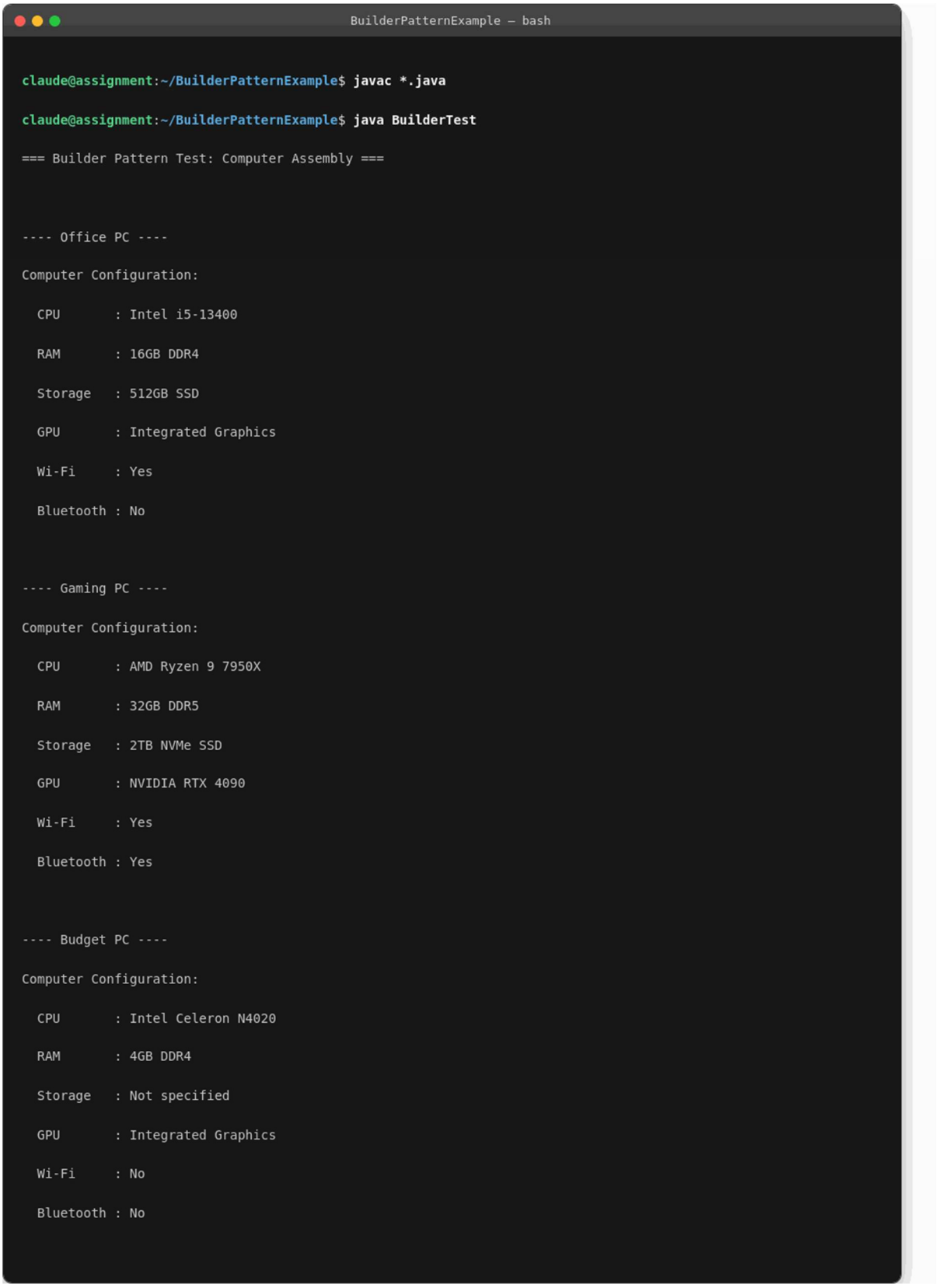

Figure 3: Compilation and execution output - BuilderPatternExample

Output / Result

The console output lists three distinct Computer configurations - Office PC, Gaming PC, and Budget PC - each assembled through the same fluent Builder, with optional fields (Storage, GPU, Wi-Fi, Bluetooth) correctly defaulted when not specified. This confirms the Builder correctly manages step-by-step construction of varied object configurations.

Exercise 4: Implementing the Adapter Pattern

Pattern Category: Structural Pattern

Aim: To implement the Adapter design pattern in Java for a payment processing system that must integrate multiple third-party gateways exposing different interfaces.

Concept: Converts the interface of a class into another interface that clients expect, allowing otherwise-incompatible classes to work together.

Scenario: A payment processing system must integrate with multiple third-party payment gateways, each exposing a different, incompatible method signature. The Adapter pattern is used to make these gateways usable through one common interface.

Steps Performed

1. Created a new Java project named AdapterPatternExample.
2. Defined the target interface PaymentProcessor with a processPayment(double amount) method.
3. Implemented two adaptee classes representing real third-party gateways: PayPalGateway (with method sendPayment(double)) and StripeGateway (with method charge(int cents, String currency)).
4. Implemented the adapter classes PayPalAdapter and StripeAdapter, each implementing PaymentProcessor and translating the unified call into the gateway-specific method.
5. Created an AdapterTest class to demonstrate processing payments through both gateways using the single, common PaymentProcessor interface.

Project / Files Created

- AdapterPatternExample/src/PaymentProcessor.java
- AdapterPatternExample/src/PayPalGateway.java
- AdapterPatternExample/src/StripeGateway.java
- AdapterPatternExample/src/PayPalAdapter.java
- AdapterPatternExample/src/StripeAdapter.java
- AdapterPatternExample/src/AdapterTest.java

Source Code

PaymentProcessor.java

```
/** PaymentProcessor.java - the Target interface expected by client code. */
public interface PaymentProcessor {
    void processPayment(double amount);
}
```

PayPalGateway.java

```
/**
 * PayPalGateway.java - Adaptee #1.
 * A third-party gateway with its OWN incompatible method signature.
 */
public class PayPalGateway {
    public void sendPayment(double amountInUSD) {
        System.out.println("[PayPal] Sending payment of $" + amountInUSD + " via PayPal
API.");
    }
}
```

StripeGateway.java

```
/**
 * StripeGateway.java - Adaptee #2.
 * Another third-party gateway, with a different incompatible interface
 * (works in cents, and needs a currency code).
 */
public class StripeGateway {
    public void charge(int amountInCents, String currency) {
        System.out.println("[Stripe] Charging " + amountInCents + " " +
currency.toUpperCase()
                + " cents via Stripe API.");
    }
}
```

PayPalAdapter.java

```
/**
 * PayPalAdapter.java
 * Adapts PayPalGateway.sendPayment(double) to the PaymentProcessor.processPayment(double)
interface.
 */
public class PayPalAdapter implements PaymentProcessor {
    private final PayPalGateway payPalGateway;

    public PayPalAdapter(PayPalGateway payPalGateway) {
        this.payPalGateway = payPalGateway;
    }

    @Override
    public void processPayment(double amount) {
        // translate the unified call into the gateway-specific call
        payPalGateway.sendPayment(amount);
    }
}
```

StripeAdapter.java

```
/**
 * StripeAdapter.java
 * Adapts StripeGateway.charge(int cents, String currency) to processPayment(double).
 */
```

```
public class StripeAdapter implements PaymentProcessor {
    private final StripeGateway stripeGateway;

    public StripeAdapter(StripeGateway stripeGateway) {
        this.stripeGateway = stripeGateway;
    }

    @Override
    public void processPayment(double amount) {
        int cents = (int) Math.round(amount * 100);
        stripeGateway.charge(cents, "usd");
    }
}
```

AdapterTest.java

```
/**
 * AdapterTest.java
 * Shows that totally different payment gateway APIs can be used
 * interchangeably through the common PaymentProcessor interface.
 */
public class AdapterTest {
    public static void main(String[] args) {
        System.out.println("=== Adapter Pattern Test: Payment Gateways ===\n");

        PaymentProcessor[] processors = {
            new PayPalAdapter(new PayPalGateway()),
            new StripeAdapter(new StripeGateway())
        };

        double[] amounts = { 49.99, 120.50 };

        for (int i = 0; i < processors.length; i++) {
            System.out.println("Processor: " + processors[i].getClass().getSimpleName());
            processors[i].processPayment(amounts[i]);
            System.out.println("------------------------------------------------------");
        }

        System.out.println("\nBoth gateways were called through the SAME PaymentProcessor");
        System.out.println("interface, even though their native APIs are incompatible.");
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and run from the command line as shown below: "javac *.java" followed by "java AdapterTest". The real console output is captured in the screenshot below.

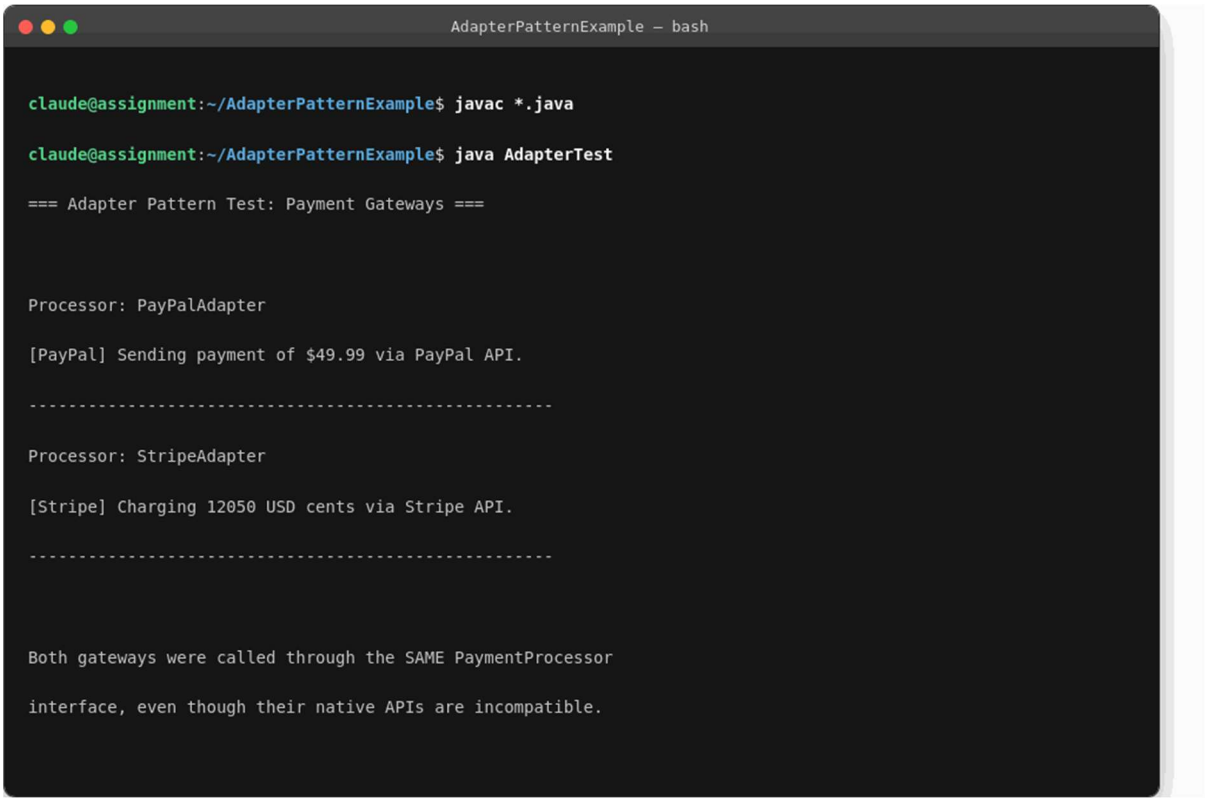

Figure 4: Compilation and execution output - AdapterPatternExample

Output / Result

The output shows PayPalAdapter and StripeAdapter both being invoked through the identical processPayment(amount) call, even though PayPalGateway natively expects sendPayment(double) and StripeGateway natively expects charge(int, String). This confirms the Adapter pattern successfully bridges incompatible third-party interfaces.

Exercise 5: Implementing the Decorator Pattern

Pattern Category: Structural Pattern

Aim: To implement the Decorator design pattern in Java for a notification system that can send notifications through multiple channels added dynamically.

Concept: Attaches additional responsibilities to an object dynamically, providing a flexible alternative to subclassing for extending behaviour.

Scenario: A notification system must be able to send notifications via multiple channels (Email, SMS, Slack). The Decorator pattern is used so additional channels can be attached to a base notifier dynamically, without altering its source code.

Steps Performed

1. Created a new Java project named DecoratorPatternExample.
2. Defined the Notifier component interface with a send(String message) method.
3. Implemented the concrete component EmailNotifier, which implements Notifier.
4. Implemented the abstract decorator class NotifierDecorator, which implements Notifier and holds a reference to a wrapped Notifier object.
5. Created the concrete decorator classes SMSNotifierDecorator and SlackNotifierDecorator, both extending NotifierDecorator.
6. Created a DecoratorTest class to demonstrate sending notifications through Email only, Email+SMS, and Email+SMS+Slack by stacking decorators at runtime.

Project / Files Created

- DecoratorPatternExample/src/Notifier.java
- DecoratorPatternExample/src/EmailNotifier.java
- DecoratorPatternExample/src/NotifierDecorator.java
- DecoratorPatternExample/src/SMSNotifierDecorator.java
- DecoratorPatternExample/src/SlackNotifierDecorator.java
- DecoratorPatternExample/src/DecoratorTest.java

Source Code

Notifier.java

```
/** Notifier.java - Component interface for the notification system. */
public interface Notifier {
    void send(String message);
}
```

EmailNotifier.java

```
/** EmailNotifier.java - Concrete Component: the base notification channel. */
public class EmailNotifier implements Notifier {
    @Override
    public void send(String message) {
        System.out.println("[Email] Sending message: \"" + message + "\"");
    }
}
```

NotifierDecorator.java

```
/**
 * NotifierDecorator.java
 * Abstract Decorator that implements Notifier and wraps another Notifier,
 * so additional channels can be stacked dynamically at runtime.
 */
public abstract class NotifierDecorator implements Notifier {
    protected final Notifier wrappedNotifier;

    public NotifierDecorator(Notifier notifier) {
        this.wrappedNotifier = notifier;
    }

    @Override
    public void send(String message) {
        wrappedNotifier.send(message); // always forward to the wrapped object first
    }
}
```

SMSNotifierDecorator.java

```
/** SMSNotifierDecorator.java - Concrete Decorator adding SMS notifications. */
public class SMSNotifierDecorator extends NotifierDecorator {
    public SMSNotifierDecorator(Notifier notifier) { super(notifier); }

    @Override
    public void send(String message) {
        super.send(message);
        System.out.println("[SMS] Sending message: \"" + message + "\"");
    }
}
```

SlackNotifierDecorator.java

```
/** SlackNotifierDecorator.java - Concrete Decorator adding Slack notifications. */
public class SlackNotifierDecorator extends NotifierDecorator {
    public SlackNotifierDecorator(Notifier notifier) { super(notifier); }

    @Override
    public void send(String message) {
        super.send(message);
        System.out.println("[Slack] Posting message to #alerts: \"" + message + "\"");
```

```
    }
}
```

DecoratorTest.java

```
/**
 * DecoratorTest.java
 * Demonstrates stacking extra notification channels onto a base
 * EmailNotifier at runtime, without modifying EmailNotifier itself.
 */
public class DecoratorTest {
    public static void main(String[] args) {
        System.out.println("=== Decorator Pattern Test: Notification System ===\n");

        System.out.println("-- Plain Email only --");
        Notifier emailOnly = new EmailNotifier();
        emailOnly.send("Server CPU usage exceeded 90%.");

        System.out.println("\n-- Email + SMS --");
        Notifier emailAndSms = new SMSNotifierDecorator(new EmailNotifier());
        emailAndSms.send("Scheduled maintenance starts in 1 hour.");

        System.out.println("\n-- Email + SMS + Slack (all channels stacked) --");
        Notifier allChannels = new SlackNotifierDecorator(new SMSNotifierDecorator(new
EmailNotifier()));
        allChannels.send("Production deployment completed successfully.");

        System.out.println("\nNotification behaviour was extended dynamically using
decorators,");
        System.out.println("without changing the EmailNotifier class at all.");
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and run from the command line as shown below: "javac *.java" followed by "java DecoratorTest". The real console output is captured in the screenshot below.

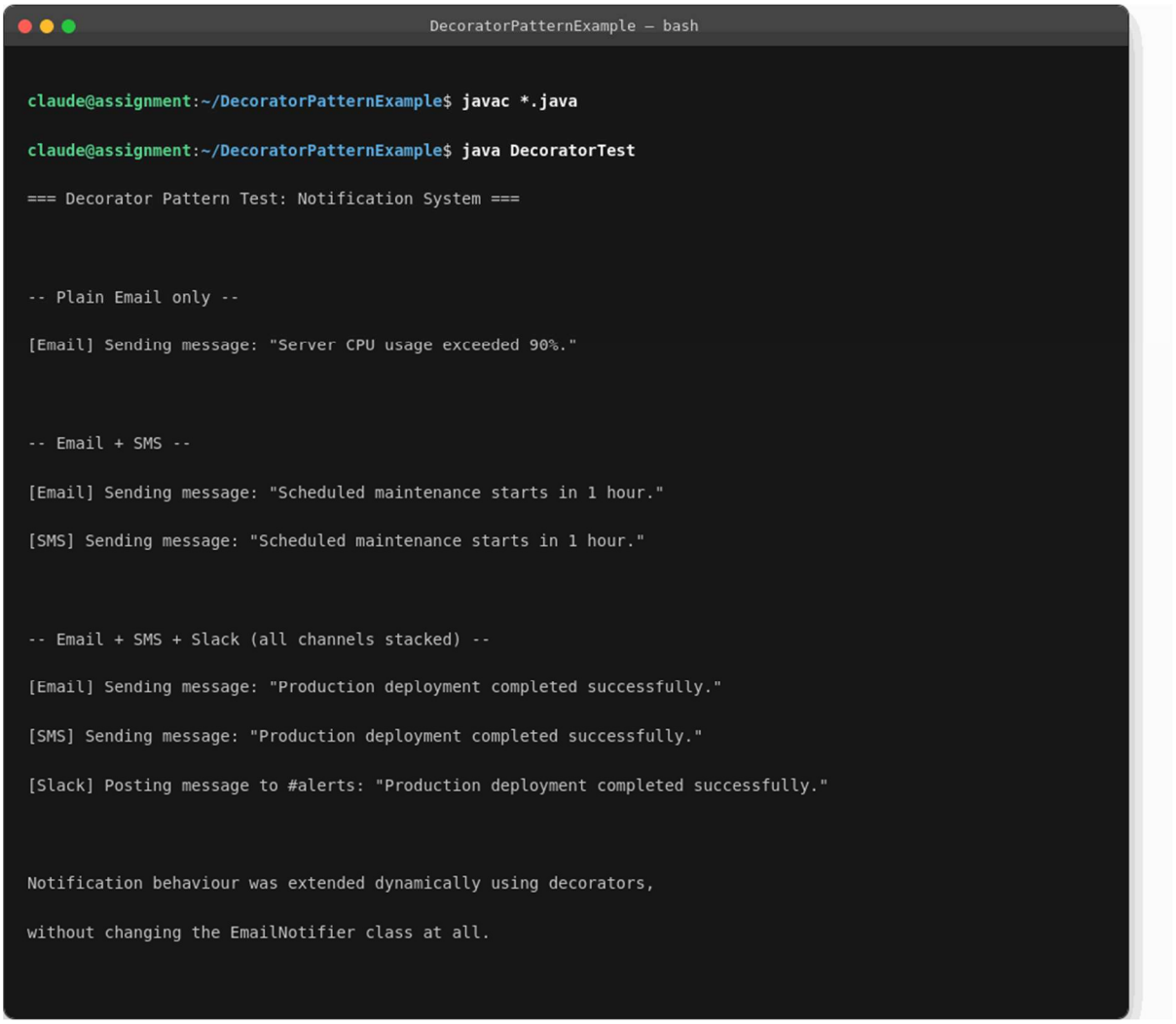

Figure 5: Compilation and execution output - DecoratorPatternExample

Output / Result

The output shows that the same base EmailNotifier gained SMS, and then Slack, notification behaviour purely by being wrapped in decorators at runtime - the EmailNotifier class itself was never modified. This confirms the Decorator pattern's ability to extend behaviour dynamically and compose channels freely.

Exercise 6: Implementing the Proxy Pattern

Pattern Category: Structural Pattern

Aim: To implement the Proxy design pattern in Java for an image viewer application that loads images from a remote server, adding lazy initialization and caching.

Concept: Provides a surrogate or placeholder for another object in order to control access to it (e.g., lazy loading, caching, access control).

Scenario: An image viewer application loads images from a remote server, which is an expensive operation. The Proxy pattern is used to delay that download until the image is actually needed (lazy initialization), and to cache it for subsequent requests.

Steps Performed

1. Created a new Java project named ProxyPatternExample.
2. Defined the Image subject interface with a display() method.
3. Implemented the RealImage class, which implements Image and simulates downloading the image from a remote server inside its constructor.
4. Implemented the ProxyImage class, which implements Image, holds a reference to a RealImage, and only creates that RealImage the first time display() is called.
5. Implemented caching inside ProxyImage so subsequent calls reuse the already-loaded RealImage instead of downloading again.
6. Created a ProxyTest class to demonstrate calling display() three times and confirming the remote download happens only once.

Project / Files Created

- ProxyPatternExample/src/Image.java
- ProxyPatternExample/src/RealImage.java
- ProxyPatternExample/src/ProxyImage.java
- ProxyPatternExample/src/ProxyTest.java

Source Code

Image.java

```
/** Image.java - Subject interface implemented by both the real image and its proxy. */
public interface Image {
    void display();
}
```

RealImage.java

```
/**
 * RealImage.java - Real Subject.
 * Simulates an expensive operation: downloading an image from a remote server.
 * The download happens once, inside the constructor.
 */
public class RealImage implements Image {
    private final String fileName;

    public RealImage(String fileName) {
        this.fileName = fileName;
        loadFromRemoteServer();
    }

    private void loadFromRemoteServer() {
        System.out.println("[RealImage] Downloading \"" + fileName + "\" from remote
server... (expensive operation)");
    }

    @Override
    public void display() {
        System.out.println("[RealImage] Displaying \"" + fileName + "\".");
    }
}
```

ProxyImage.java

```
/**
 * ProxyImage.java - Proxy.
 * Implements lazy initialization (the RealImage / remote download only happens
 * the first time display() is actually called) and simple caching afterwards.
 */
public class ProxyImage implements Image {
    private final String fileName;
    private RealImage realImage; // not created until needed

    public ProxyImage(String fileName) {
        this.fileName = fileName;
        System.out.println("[ProxyImage] Proxy created for \"" + fileName + "\" (no download
yet).");
    }

    @Override
    public void display() {
        if (realImage == null) {
            System.out.println("[ProxyImage] First request - triggering lazy load.");
            realImage = new RealImage(fileName); // download + cache happens only once
        } else {
            System.out.println("[ProxyImage] Serving \"" + fileName + "\" from cache - no
download needed.");
        }
        realImage.display();
    }
}
```

ProxyTest.java

```
/**
 * ProxyTest.java
 * Demonstrates that the image is only downloaded once (lazy initialization)
 * even though display() is called multiple times (caching).
 */
public class ProxyTest {
    public static void main(String[] args) {
        System.out.println("=== Proxy Pattern Test: Image Viewer ===\n");

        Image photo = new ProxyImage("vacation_photo.jpg");

        System.out.println("\n-- Calling display() for the 1st time --");
        photo.display();

        System.out.println("\n-- Calling display() for the 2nd time --");
        photo.display();

        System.out.println("\n-- Calling display() for the 3rd time --");
        photo.display();

        System.out.println("\nNotice the remote download happened only ONCE,");
        System.out.println("on the first call, thanks to the Proxy's lazy loading +
caching.");
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and run from the command line as shown below: "javac *.java" followed by "java ProxyTest". The real console output is captured in the screenshot below.

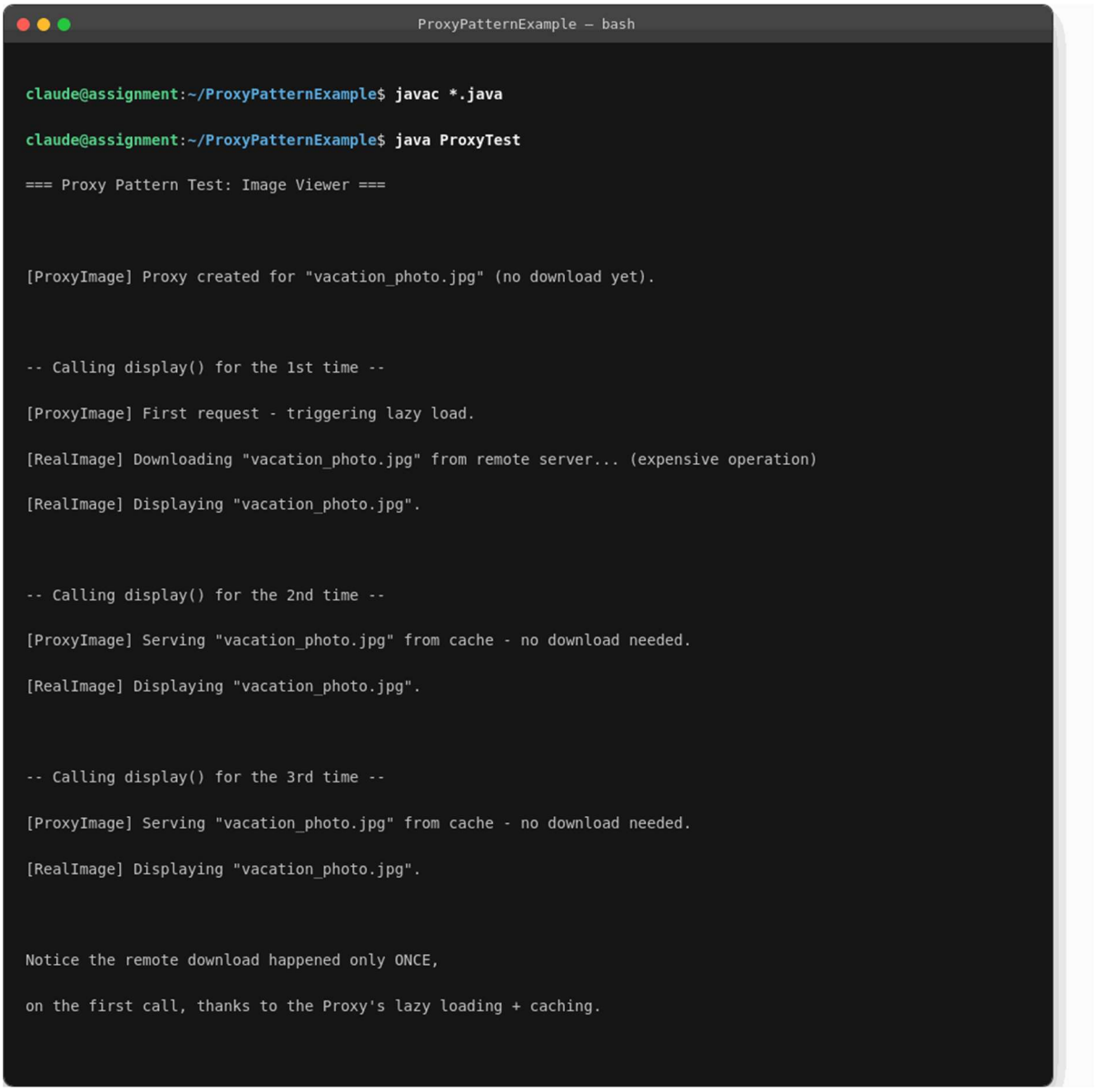

Figure 6: Compilation and execution output - ProxyPatternExample

Output / Result

The console output shows the “Downloading ... from remote server” message appearing only once, on the first display() call, while the 2nd and 3rd calls print “Serving ... from cache.” This confirms that ProxyImage correctly implements lazy initialization and caching, exactly as the Proxy pattern requires.

Exercise 7: Implementing the Observer Pattern

Pattern Category: Behavioral Pattern

Aim: To implement the Observer design pattern in Java for a stock market monitoring application where multiple clients must be notified whenever a stock price changes.

Concept: Defines a one-to-many dependency so that when one object changes state, all of its dependents are notified automatically.

Scenario: A stock market monitoring application must notify multiple client applications (mobile, web) whenever a stock's price changes. The Observer pattern is used to let clients subscribe to and receive these updates automatically.

Steps Performed

1. Created a new Java project named ObserverPatternExample.
2. Defined the Stock subject interface with registerObserver(), deregisterObserver(), and notifyObservers() methods.
3. Implemented the concrete subject StockMarket, which maintains an internal list of observers and notifies them whenever a price is set.
4. Defined the Observer interface with an update(String stockSymbol, double price) method.
5. Implemented the concrete observers MobileApp and WebApp, both implementing Observer.
6. Created an ObserverTest class to demonstrate registering both observers, broadcasting price changes, deregistering one observer, and broadcasting a further change.

Project / Files Created

- ObserverPatternExample/src/Stock.java
- ObserverPatternExample/src/StockMarket.java
- ObserverPatternExample/src/Observer.java
- ObserverPatternExample/src/MobileApp.java
- ObserverPatternExample/src/WebApp.java
- ObserverPatternExample/src/ObserverTest.java

Source Code

Stock.java

```
/** Stock.java - Subject interface: register/deregister/notify observers. */
public interface Stock {
    void registerObserver(Observer observer);
    void deregisterObserver(Observer observer);
    void notifyObservers();
}
```

StockMarket.java

```
import java.util.ArrayList;
import java.util.List;

/**
 * StockMarket.java - Concrete Subject.
 * Maintains a list of Observers and notifies all of them whenever a
 * stock price changes.
 */
public class StockMarket implements Stock {
    private final List<Observer> observers = new ArrayList<>();
    private String stockSymbol;
    private double price;

    @Override
    public void registerObserver(Observer observer) {
        observers.add(observer);
        System.out.println("[StockMarket] " + observer.getClass().getSimpleName() + "
registered for updates.");
    }

    @Override
    public void deregisterObserver(Observer observer) {
        observers.remove(observer);
        System.out.println("[StockMarket] " + observer.getClass().getSimpleName() + "
deregistered.");
    }

    @Override
    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(stockSymbol, price);
        }
    }

    // Called whenever the price changes - triggers notification to all observers
    public void setStockPrice(String stockSymbol, double price) {
        this.stockSymbol = stockSymbol;
        this.price = price;
        System.out.println("\n[StockMarket] " + stockSymbol + " price changed to $" +
price);
        notifyObservers();
    }
}
```

Observer.java

```
/** Observer.java - implemented by every client that wants stock updates. */
public interface Observer {
    void update(String stockSymbol, double price);
}
```

MobileApp.java

```
public class MobileApp implements Observer {
    @Override
    public void update(String stockSymbol, double price) {
        System.out.println("  [MobileApp] Push notification: " + stockSymbol + " is now $" +
price);
    }
}
```

WebApp.java

```
public class WebApp implements Observer {
    @Override
    public void update(String stockSymbol, double price) {
        System.out.println("  [WebApp] Dashboard updated: " + stockSymbol + " is now $" +
price);
    }
}
```

ObserverTest.java

```
/**
 * ObserverTest.java
 * Demonstrates registering/deregistering observers and broadcasting
 * stock price changes to all currently registered observers.
 */
public class ObserverTest {
    public static void main(String[] args) {
        System.out.println("=== Observer Pattern Test: Stock Market Monitor ===");

        StockMarket market = new StockMarket();
        Observer mobile = new MobileApp();
        Observer web = new WebApp();

        market.registerObserver(mobile);
        market.registerObserver(web);

        market.setStockPrice("AAPL", 195.32);
        market.setStockPrice("GOOG", 2840.15);

        System.out.println("\n[Test] Deregistering MobileApp...");
        market.deregisterObserver(mobile);

        market.setStockPrice("AAPL", 198.10);

        System.out.println("\nOnly currently-registered observers (WebApp) received the
final update.");
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and run from the command line as shown below: "javac *.java" followed by "java ObserverTest". The real console output is captured in the screenshot below.

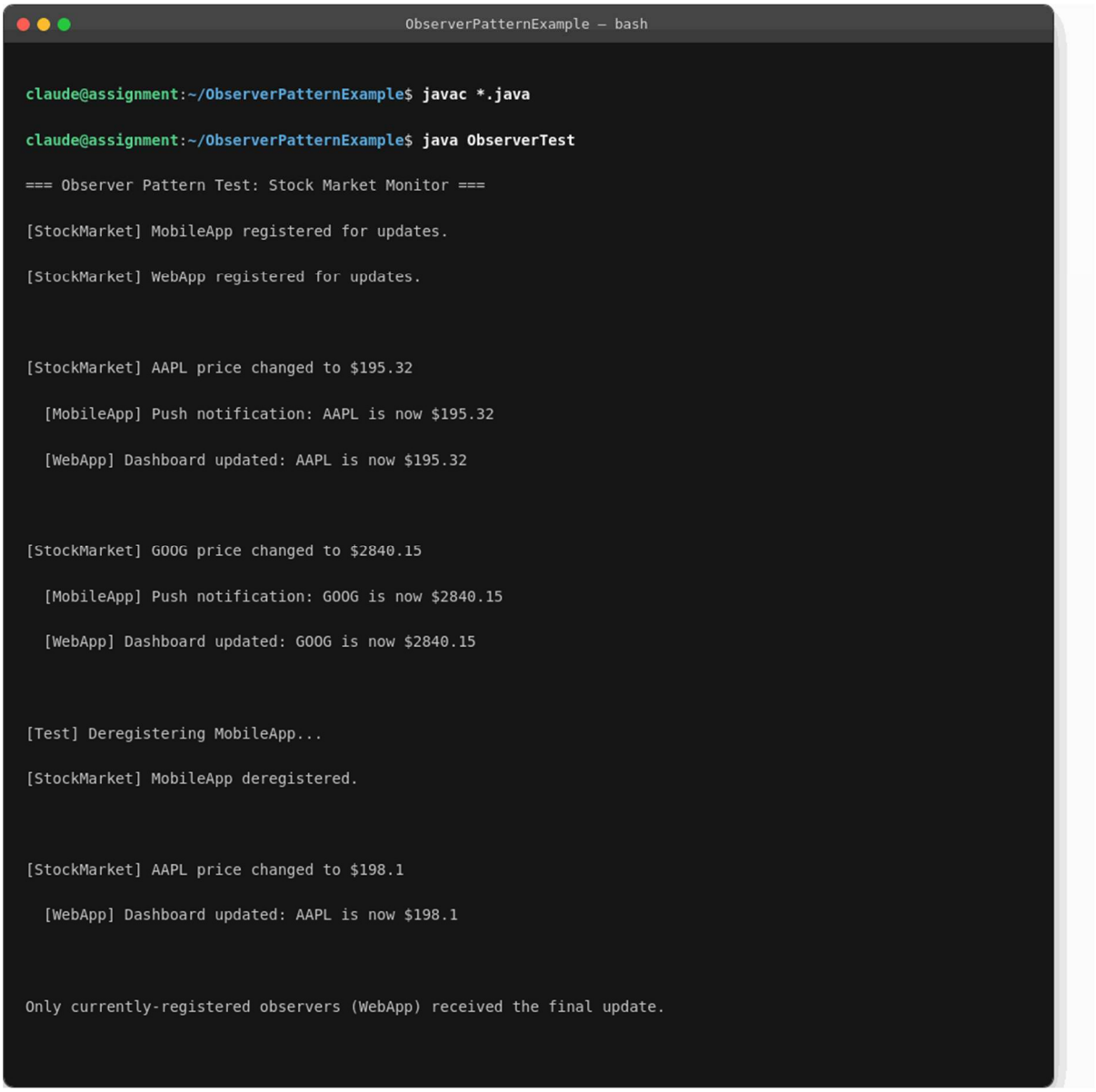

Figure 7: Compilation and execution output - ObserverPatternExample

Output / Result

The output shows both MobileApp and WebApp receiving the AAPL and GOOG price updates while registered. After MobileApp is deregistered, only WebApp receives the final AAPL update - confirming that StockMarket correctly maintains and notifies its current list of observers, and respects deregistration.

Exercise 8: Implementing the Strategy Pattern

Pattern Category: Behavioral Pattern

Aim: To implement the Strategy design pattern in Java for a payment system where different payment methods can be selected at runtime.

Concept: Defines a family of interchangeable algorithms and lets the algorithm vary independently from the clients that use it.

Scenario: A payment system must allow different payment methods (Credit Card, PayPal) to be selected at runtime, without the client code branching on the chosen method using if/else or switch statements.

Steps Performed

1. Created a new Java project named StrategyPatternExample.
2. Defined the PaymentStrategy interface with a pay(double amount) method.
3. Implemented the concrete strategies CreditCardPayment and PayPalPayment, each implementing PaymentStrategy.
4. Implemented the PaymentContext class, which holds a reference to a PaymentStrategy and exposes an executePayment() method that delegates to the current strategy.
5. Created a StrategyTest class to demonstrate selecting CreditCardPayment, executing a payment, then switching to PayPalPayment and executing another payment through the same context.

Project / Files Created

- StrategyPatternExample/src/PaymentStrategy.java
- StrategyPatternExample/src/CreditCardPayment.java
- StrategyPatternExample/src/PayPalPayment.java
- StrategyPatternExample/src/PaymentContext.java
- StrategyPatternExample/src/StrategyTest.java

Source Code

PaymentStrategy.java

```
/** PaymentStrategy.java - Strategy interface, one implementation per payment method. */
public interface PaymentStrategy {
    void pay(double amount);
}
```

CreditCardPayment.java

```
public class CreditCardPayment implements PaymentStrategy {
    private final String cardNumber;
```

```
    public CreditCardPayment(String cardNumber) { this.cardNumber = cardNumber; }

    @Override
    public void pay(double amount) {
        String masked = "**** **** **** " + cardNumber.substring(cardNumber.length() - 4);
        System.out.println("[CreditCard] Charged $" + amount + " to card " + masked);
    }
}
```

PayPalPayment.java

```
public class PayPalPayment implements PaymentStrategy {
    private final String email;

    public PayPalPayment(String email) { this.email = email; }

    @Override
    public void pay(double amount) {
        System.out.println("[PayPal] Charged $" + amount + " to PayPal account " + email);
    }
}
```

PaymentContext.java

```
/**
 * PaymentContext.java
 * Holds a reference to a PaymentStrategy and delegates the actual
 * payment work to whichever strategy is currently set - selectable at runtime.
 */
public class PaymentContext {
    private PaymentStrategy strategy;

    public void setPaymentStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
        System.out.println("[PaymentContext] Strategy switched to: " +
strategy.getClass().getSimpleName());
    }

    public void executePayment(double amount) {
        if (strategy == null) {
            throw new IllegalStateException("No payment strategy set.");
        }
        strategy.pay(amount);
    }
}
```

StrategyTest.java

```
/**
 * StrategyTest.java
 * Demonstrates selecting different payment strategies at runtime
 * through the same PaymentContext object.
```

```
 */
public class StrategyTest {
    public static void main(String[] args) {
        System.out.println("=== Strategy Pattern Test: Payment System ===\n");

        PaymentContext context = new PaymentContext();

        context.setPaymentStrategy(new CreditCardPayment("4111111111111234"));
        context.executePayment(149.99);

        System.out.println();
        context.setPaymentStrategy(new PayPalPayment("buyer@example.com"));
        context.executePayment(75.50);

        System.out.println("\nThe SAME PaymentContext processed two different payment");
        System.out.println("methods simply by swapping the strategy at runtime.");
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and run from the command line as shown below: "javac *.java" followed by "java StrategyTest". The real console output is captured in the screenshot below.

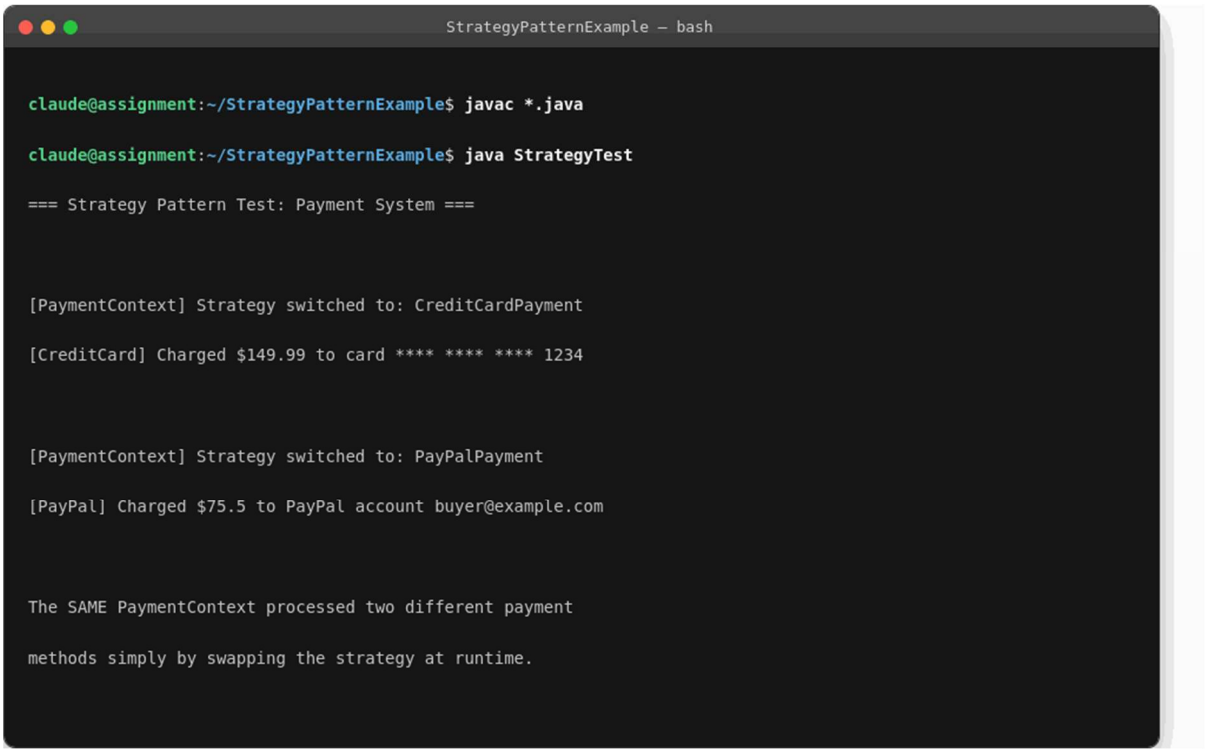

Figure 8: Compilation and execution output - StrategyPatternExample

Output / Result

The output shows the same PaymentContext object successfully executing two different payment algorithms - first CreditCardPayment, then PayPalPayment - simply by swapping the injected strategy object. This confirms the Strategy pattern allows the payment algorithm to vary independently of the client using it.

Exercise 9: Implementing the Command Pattern

Pattern Category: Behavioral Pattern

Aim: To implement the Command design pattern in Java for a home automation system in which commands can be issued to turn devices on or off.

Concept: Encapsulates a request as an object, allowing parameterization of clients with different requests, queuing, and decoupling of sender from receiver.

Scenario: A home automation system must be able to issue commands to turn devices on or off, without the remote control needing to know how each device performs that action. The Command pattern is used to encapsulate each action as an object.

Steps Performed

1. Created a new Java project named CommandPatternExample.
2. Defined the Command interface with an execute() method.
3. Implemented the concrete commands LightOnCommand and LightOffCommand, each implementing Command.
4. Implemented the RemoteControl invoker class, which holds a reference to a Command and triggers it via pressButton().
5. Implemented the Light receiver class, with turnOn() and turnOff() methods that perform the actual work.
6. Created a CommandTest class to demonstrate issuing on/off commands to two different Light receivers through the same RemoteControl.

Project / Files Created

- CommandPatternExample/src/Command.java
- CommandPatternExample/src/Light.java
- CommandPatternExample/src/LightOnCommand.java
- CommandPatternExample/src/LightOffCommand.java
- CommandPatternExample/src/RemoteControl.java
- CommandPatternExample/src/CommandTest.java

Source Code

Command.java

```
/** Command.java - Command interface, encapsulating an action as an object. */
public interface Command {
    void execute();
}
```

Light.java

```
/** Light.java - Receiver: knows how to actually perform the requested actions. */
public class Light {
    private final String location;

    public Light(String location) { this.location = location; }

    public void turnOn() { System.out.println("[Light] " + location + " light is now ON.");
}
    public void turnOff() { System.out.println("[Light] " + location + " light is now
OFF."); }
}
```

LightOnCommand.java

```
/** LightOnCommand.java - Concrete Command binding a Light receiver to the "on" action. */
public class LightOnCommand implements Command {
    private final Light light;

    public LightOnCommand(Light light) { this.light = light; }

    @Override
    public void execute() { light.turnOn(); }
}
```

LightOffCommand.java

```
/** LightOffCommand.java - Concrete Command binding a Light receiver to the "off" action. */
public class LightOffCommand implements Command {
    private final Light light;

    public LightOffCommand(Light light) { this.light = light; }

    @Override
    public void execute() { light.turnOff(); }
}
```

RemoteControl.java

```
/**
 * RemoteControl.java - Invoker.
 * Holds a reference to a Command and triggers it, without knowing
 * anything about what the command actually does or which Receiver it uses.
 */
public class RemoteControl {
    private Command command;

    public void setCommand(Command command) {
        this.command = command;
    }
```

```
    public void pressButton() {
        if (command == null) {
            System.out.println("[RemoteControl] No command set.");
            return;
        }
        command.execute();
    }
}
```

CommandTest.java

```
/**
 * CommandTest.java
 * Demonstrates issuing different commands to different devices
 * through the same RemoteControl invoker.
 */
public class CommandTest {
    public static void main(String[] args) {
        System.out.println("=== Command Pattern Test: Home Automation ===\n");

        Light livingRoomLight = new Light("Living Room");
        Light bedroomLight = new Light("Bedroom");

        Command livingRoomOn = new LightOnCommand(livingRoomLight);
        Command livingRoomOff = new LightOffCommand(livingRoomLight);
        Command bedroomOn = new LightOnCommand(bedroomLight);
        Command bedroomOff = new LightOffCommand(bedroomLight);

        RemoteControl remote = new RemoteControl();

        System.out.println("-- Turning ON living room light --");
        remote.setCommand(livingRoomOn);
        remote.pressButton();

        System.out.println("\n-- Turning ON bedroom light --");
        remote.setCommand(bedroomOn);
        remote.pressButton();

        System.out.println("\n-- Turning OFF living room light --");
        remote.setCommand(livingRoomOff);
        remote.pressButton();

        System.out.println("\n-- Turning OFF bedroom light --");
        remote.setCommand(bedroomOff);
        remote.pressButton();

        System.out.println("\nThe SAME RemoteControl issued four different commands");
        System.out.println("to two different Light receivers, fully decoupled.");
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and run from the command line as shown below: "javac *.java" followed by "java CommandTest". The real console output is captured in the screenshot below.

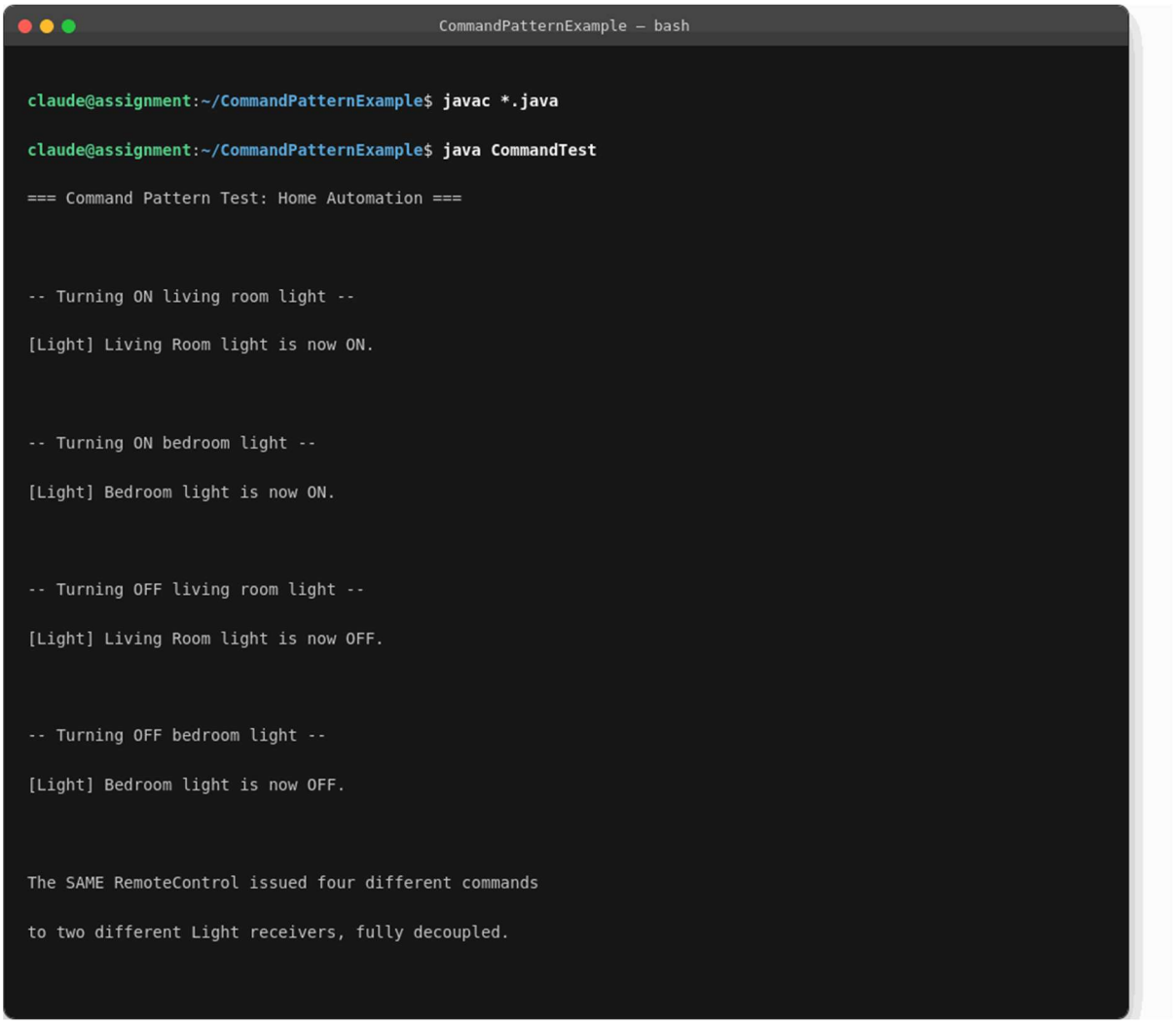

Figure 9: Compilation and execution output - CommandPatternExample

Output / Result

The output shows the same RemoteControl object correctly executing four different commands (living room ON, bedroom ON, living room OFF, bedroom OFF) across two different Light receivers, with no conditional logic inside RemoteControl about which device or action to run. This confirms the Command pattern decouples the invoker from the receiver.

Exercise 10: Implementing the MVC Pattern

Pattern Category: Architectural Pattern

Aim: To implement the Model-View-Controller (MVC) pattern in Java for a simple application that manages student records.

Concept: Separates an application into three interconnected components - Model (data), View (presentation), and Controller (logic) - to isolate concerns.

Scenario: A simple web-style application is needed to manage student records using the MVC pattern, keeping data storage, presentation, and control logic clearly separated.

Steps Performed

1. Created a new Java project named MVCPatternExample.
2. Defined the Student model class with attributes name, id, and grade.
3. Defined the StudentView class with a displayStudentDetails() method responsible only for presentation.
4. Defined the StudentController class, which mediates between the Student model and the StudentView and exposes methods to read and update student data.
5. Created an MVCMainTest class to demonstrate creating a Student, updating its name and grade through StudentController, and displaying the record before and after through StudentView.

Project / Files Created

- MVCPatternExample/src/Student.java
- MVCPatternExample/src/StudentView.java
- MVCPatternExample/src/StudentController.java
- MVCPatternExample/src/MVCMainTest.java

Source Code

Student.java

```
/** Student.java - Model: pure data, no UI or control logic. */
public class Student {
    private String name;
    private String id;
    private String grade;

    public Student(String name, String id, String grade) {
        this.name = name;
        this.id = id;
        this.grade = grade;
    }

    public String getName() { return name; }
```

```
    public void setName(String name) { this.name = name; }

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getGrade() { return grade; }
    public void setGrade(String grade) { this.grade = grade; }
}
```

StudentView.java

```
/** StudentView.java - View: only responsible for displaying data, no logic. */
public class StudentView {
    public void displayStudentDetails(String name, String id, String grade) {
        System.out.println("-----------------------------------------");
        System.out.println(" STUDENT RECORD");
        System.out.println("-----------------------------------------");
        System.out.println(" ID    : " + id);
        System.out.println(" Name  : " + name);
        System.out.println(" Grade : " + grade);
        System.out.println("-----------------------------------------");
    }
}
```

StudentController.java

```
/**
 * StudentController.java - Controller.
 * Mediates between the Model (Student) and the View (StudentView):
 * it reads/updates the model and asks the view to render it.
 */
public class StudentController {
    private final Student model;
    private final StudentView view;

    public StudentController(Student model, StudentView view) {
        this.model = model;
        this.view = view;
    }

    public void setStudentName(String name) { model.setName(name); }
    public String getStudentName() { return model.getName(); }

    public void setStudentGrade(String grade) { model.setGrade(grade); }
    public String getStudentGrade() { return model.getGrade(); }

    public void updateView() {
        view.displayStudentDetails(model.getName(), model.getId(), model.getGrade());
    }
}
```

MVCMainTest.java

```
/**
 * MVCMainTest.java
 * Demonstrates creating a Student, updating it through the Controller,
 * and rendering it through the View - the three MVC layers stay decoupled.
 */
public class MVCMainTest {
    public static void main(String[] args) {
        System.out.println("=== MVC Pattern Test: Student Record Management ===\n");

        Student model = new Student("Asha Rao", "STU-2026-014", "B+");
        StudentView view = new StudentView();
        StudentController controller = new StudentController(model, view);

        System.out.println("-- Initial record --");
        controller.updateView();

        System.out.println("\n-- Updating grade via Controller (B+  ->  A-) --");
        controller.setStudentGrade("A-");

        System.out.println("\n-- Updating name via Controller (typo fix) --");
        controller.setStudentName("Asha N. Rao");

        System.out.println("\n-- Record after updates --");
        controller.updateView();

        System.out.println("\nThe Model never talked to the View directly; the Controller");
        System.out.println("mediated every read and update, as required by the MVC
pattern.");
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and run from the command line as shown below: "javac *.java" followed by "java MVCMainTest". The real console output is captured in the screenshot below.

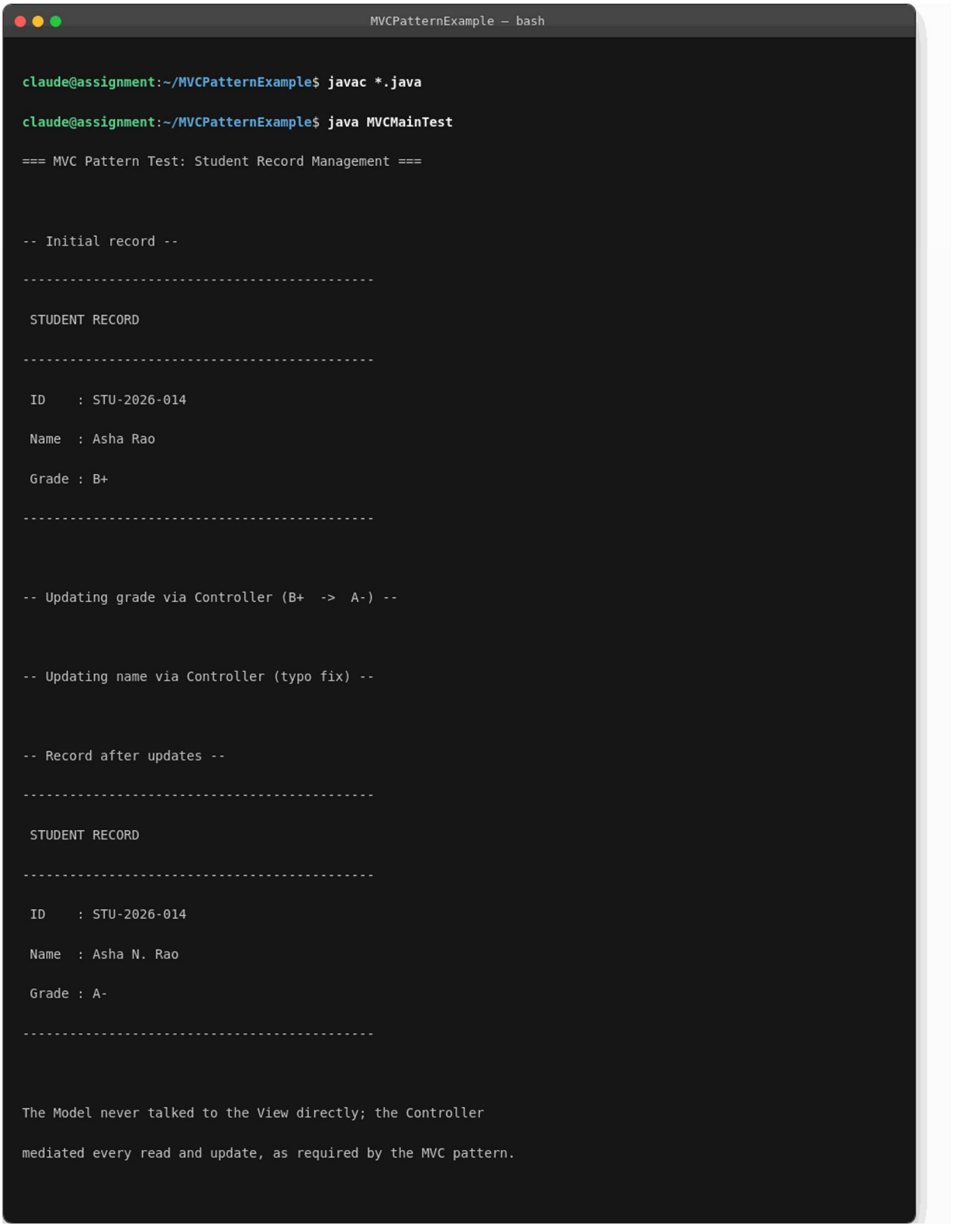

Figure 10: Compilation and execution output - MVCPatternExample

Output / Result

The output shows the initial student record, followed by the updated record (grade changed B+ to A-, name corrected), all rendered exclusively through StudentView and modified exclusively through StudentController. The Student model never communicated directly with StudentView, confirming proper separation of concerns under MVC.

Exercise 11: Implementing Dependency Injection

Pattern Category: Design Principle (Inversion of Control)

Aim: To implement Dependency Injection in Java for a customer management application where the service class depends on a repository class.

Concept: A technique whereby an object's dependencies are supplied by an external source rather than created by the object itself, reducing coupling and improving testability.

Scenario: A customer management application has a service class that depends on a repository class for data access. Dependency Injection is used to manage this dependency so the service stays decoupled from the concrete repository implementation.

Steps Performed

1. Created a new Java project named DependencyInjectionExample.
2. Defined the CustomerRepository interface with a findCustomerById(int id) method.
3. Implemented the concrete repository CustomerRepositoryImpl, which implements CustomerRepository using an in-memory data store.
4. Defined the CustomerService class, which depends on the CustomerRepository abstraction rather than a concrete class.
5. Implemented Dependency Injection using constructor injection, so CustomerRepository is passed into CustomerService's constructor from outside.
6. Created a DIMainTest class to demonstrate constructing a CustomerService with a CustomerRepositoryImpl and using it to look up several customers.

Project / Files Created

- DependencyInjectionExample/src/CustomerRepository.java
- DependencyInjectionExample/src/CustomerRepositoryImpl.java
- DependencyInjectionExample/src/CustomerService.java
- DependencyInjectionExample/src/DIMainTest.java

Source Code

CustomerRepository.java

```
/** CustomerRepository.java - abstraction the service depends on, not a concrete class. */
public interface CustomerRepository {
    String findCustomerById(int id);
}
```

CustomerRepositoryImpl.java

```
import java.util.HashMap;
import java.util.Map;

/**
 * CustomerRepositoryImpl.java
 * Concrete repository - in a real app this would query a database.
 * Here an in-memory map simulates the data store.
 */
public class CustomerRepositoryImpl implements CustomerRepository {
    private final Map<Integer, String> customers = new HashMap<>();

    public CustomerRepositoryImpl() {
        customers.put(101, "Priya Sharma");
        customers.put(102, "Rahul Mehta");
        customers.put(103, "Ananya Iyer");
    }

    @Override
    public String findCustomerById(int id) {
        System.out.println("[CustomerRepositoryImpl] Querying data store for customer id=" +
id);
        return customers.getOrDefault(id, "No customer found with id " + id);
    }
}
```

CustomerService.java

```
/**
 * CustomerService.java
 * Depends only on the CustomerRepository ABSTRACTION, never on a concrete
 * implementation - the concrete repository is injected from the outside
 * (constructor injection), which is the core idea of Dependency Injection.
 */
public class CustomerService {
    private final CustomerRepository customerRepository;

    // Step: constructor injection
    public CustomerService(CustomerRepository customerRepository) {
        this.customerRepository = customerRepository;
    }

    public String getCustomerName(int id) {
        return customerRepository.findCustomerById(id);
    }
}
```

DIMainTest.java

```
/**
 * DIMainTest.java
 * Demonstrates wiring CustomerRepositoryImpl into CustomerService via
 * constructor injection, and using the service to look up customers.
 * Because CustomerService only knows about the CustomerRepository
```

```
 * interface, a mock/fake repository could be substituted just as easily
 * for unit testing - that decoupling is the whole point of DI.
 */
public class DIMainTest {
    public static void main(String[] args) {
        System.out.println("=== Dependency Injection Test: Customer Management ===\n");

        // The dependency is created externally...
        CustomerRepository repository = new CustomerRepositoryImpl();
        // ...and INJECTED into the service via its constructor.
        CustomerService service = new CustomerService(repository);

        System.out.println("Looking up customer 101:");
        System.out.println("  -> " + service.getCustomerName(101));

        System.out.println("\nLooking up customer 102:");
        System.out.println("  -> " + service.getCustomerName(102));

        System.out.println("\nLooking up customer 999 (does not exist):");
        System.out.println("  -> " + service.getCustomerName(999));

        System.out.println("\nCustomerService never instantiated CustomerRepositoryImpl
itself;");
        System.out.println("the dependency was injected, keeping the classes loosely
coupled.");
    }
}
```

Compilation and Execution (Screenshot)

The project was compiled and run from the command line as shown below: "javac *.java" followed by "java DIMainTest". The real console output is captured in the screenshot below.

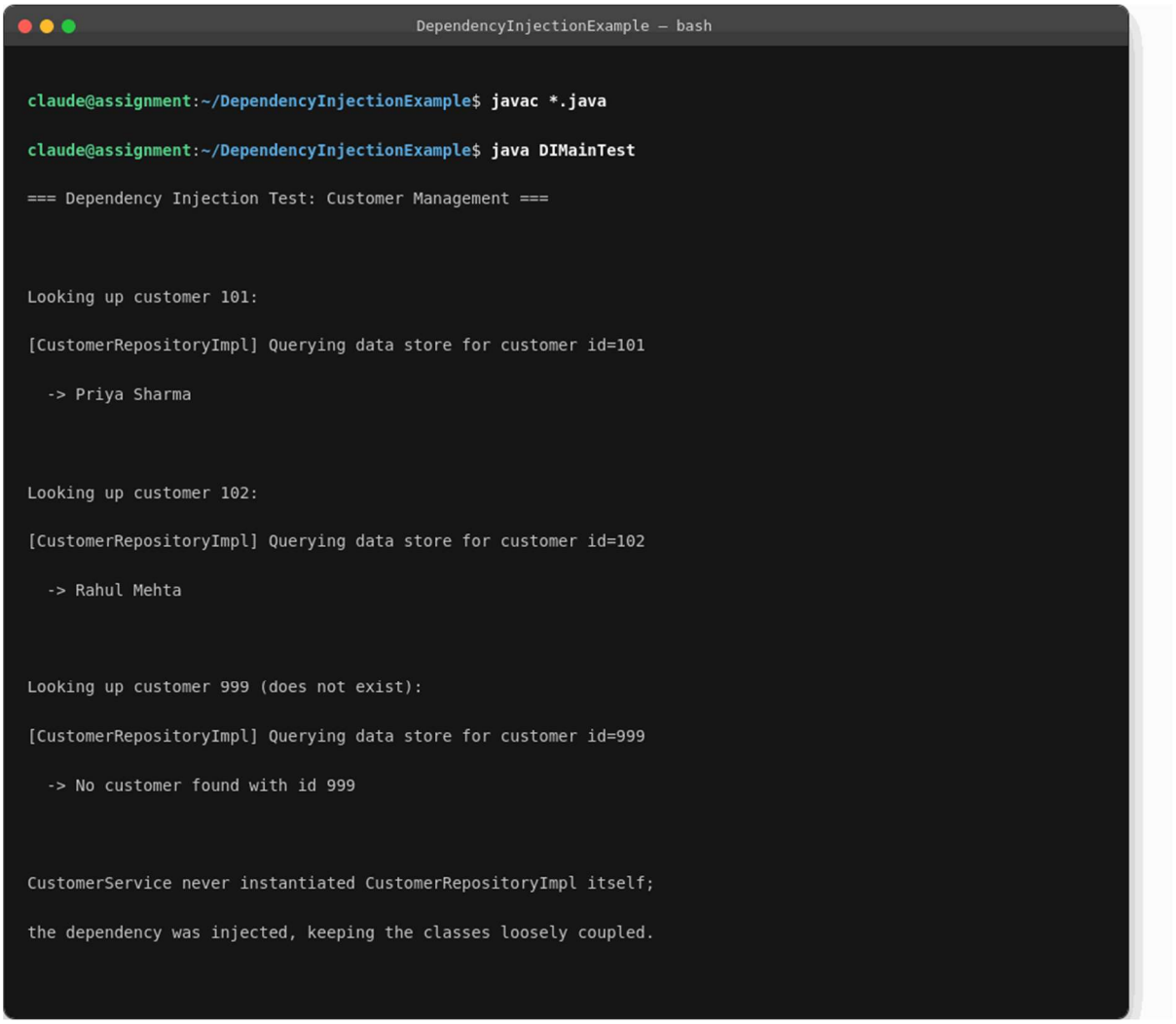

Figure 11: Compilation and execution output - DependencyInjectionExample

Output / Result

The output shows CustomerService successfully resolving customers 101, 102, and an unknown id 999, purely through the CustomerRepository reference that was injected into it. CustomerService never instantiated CustomerRepositoryImpl itself, confirming loose coupling achieved via constructor-based Dependency Injection.

Conclusion

Across these eleven exercises, the five Creational, Structural, and Behavioral Gang-of-Four pattern categories were implemented and verified hands-on in Java, alongside the MVC architectural pattern and the Dependency Injection design principle. Each implementation was independently compiled and executed, and the console output was captured as evidence that the intended behaviour - a single shared instance, decoupled object creation, flexible construction, interface translation, dynamic behaviour extension, lazy caching, automatic notification, interchangeable algorithms, decoupled command execution, separated presentation logic, and externally supplied dependencies - was achieved in each case. Together, these patterns and principles form a practical toolkit for writing Java code that is more maintainable, extensible, testable, and loosely coupled.